

Full Stack Developer Assessment: The 'KCoin' Custom Digital Currency Concept

Objective: This assignment challenges you to build a simplified, web-based simulation of a digital currency ecosystem called **KCoin**. Inspired by the foundational principles of cryptocurrencies like Bitcoin, this project will assess your ability to model complex systems, manage state across the full stack, and implement business logic in a secure and efficient manner.

Important: You are **not** expected to build a cryptographically secure, decentralized blockchain. This is a *simulation* running on a centralized server. The goal is to model the concepts of wallets, transactions, a public ledger, and a proof-of-work system within a standard client-server architecture.

Project Overview: The KCoin Ecosystem

The KCoin ecosystem has three main components:

1. **Wallets (Users):** Each user has a wallet with a unique address and a KCoin balance. Users can send KCoin to other wallet addresses.
 2. **Transactions:** When a user sends KCoin, a transaction is created and broadcast to a pool of "pending" transactions.
 3. **The Ledger (A Simplified Blockchain):** The ledger is a chain of blocks, where each block contains a list of verified transactions. To add a new block, a "miner" must solve a simple computational puzzle (Proof-of-Work). For their effort, the miner is rewarded with newly minted KCoin.
-

1. Backend Requirements (Node.js / TypeScript)

Your backend will act as the single source of truth for the entire KCoin network.

- **Data Models:** You will need models for:
 - **User (Wallet):** Should have `id`, `email/username`, `password` (hashed), and a unique `walletAddress` (can be a UUID).
 - **Transaction:** Should have `id`, `senderAddress`, `recipientAddress`, `amount`, and a `timestamp`.
 - **Block:** Should have `index`, `timestamp`, `transactions` (an array of transaction objects), `previousHash`, and a `hash`.
- **Core Logic & API Endpoints:**

- **User/Wallet Management:**
 - **POST /register:** Create a new user and generate their wallet.
 - **POST /login:** Authenticate a user (e.g., using JWT).
 - **GET /wallet:** Fetch the current user's wallet address and calculated balance (by iterating through the blockchain).
 - **Transaction Handling:**
 - **POST /transactions:** Allows an authenticated user to create a new transaction. The server should validate if the sender has sufficient funds. This new transaction is added to a temporary "pending transactions" pool.
 - **GET /transactions/pending:** Returns all transactions currently in the pending pool.
 - **Blockchain & Mining:**
 - **GET /chain:** Returns the entire blockchain (the array of all blocks).
 - **POST /mine:** This is the key endpoint. When called, the server should:
 1. Gather all transactions from the "pending" pool.
 2. Create a "coinbase" transaction, rewarding the miner with **10 KCoin**. This transaction has a special sender address like **"00"** and the recipient is the miner's wallet address.
 3. **Proof-of-Work (Simplified):** Create a new block with the pending transactions. Then, find a **nonce** (a number) which, when added to the block's content and hashed (using SHA-256), results in a hash with a predefined difficulty, e.g., **starting with "00"**.
 4. Once the correct **nonce** is found, add the new block to the chain.
 5. Clear the pending transactions pool.
 6. Return the newly created block.
-

2. Frontend Requirements (React / Next.js with TypeScript)

The frontend is the user's window into the KCoin ecosystem.

- **User Authentication:** Simple login and registration pages.
- **Dashboard:** After logging in, the user should see:
 - Their unique Wallet Address.
 - Their current KCoin Balance.
 - A form to send KCoin to another user's address.
- **Mining Interface:**
 - A simple "Mine New Block" button that calls the **POST /mine** endpoint.
 - The UI should provide feedback while mining is in progress and notify the user upon success.
- **Blockchain Explorer:**
 - A dedicated page that visualizes the entire blockchain.

- It should display each block, its hash, and the transactions contained within it. This demonstrates the public, transparent nature of the ledger.
-

Bonus Challenges (Optional)

- **Real-Time Updates:** Use WebSockets to automatically update all connected clients when a new block is mined and added to the chain.
 - **Dynamic Difficulty:** Implement logic where the Proof-of-Work difficulty (e.g., the required number of leading zeros in the hash) increases as the chain grows.
 - **Transaction Fees:** Introduce a small, optional transaction fee that miners collect as an additional incentive.
 - **UI/UX Polish:** Create a more visually engaging Blockchain Explorer, perhaps with graphical links between blocks.
-

Evaluation Criteria

- **System Modeling:** Your ability to translate the complex concepts of a cryptocurrency into a functional, simplified application.
- **Full-Stack Integration:** How seamlessly your frontend and backend communicate.
- **Code Quality:** Clean, modular, and maintainable code with effective use of TypeScript.
- **Problem-Solving:** How you approached the logic for calculating balances, validating transactions, and implementing the mining process.
- **Functionality:** The application works as described, meeting all core requirements.

Submission Instructions

1. Create a private Git repository for your project.
2. Include a **README.md** file with clear instructions on how to set up and run both the backend and frontend services locally.
3. In your README, briefly explain any architectural decisions or assumptions you made.
4. Once complete, please invite us to the repository and send us the link.
5. Provide the video recording of your demo and explanation of the code.